

Herald — CLAUDE_CODE

Herald — Claude Code Dispatcher Setup Guide

Table of contents

Pre-requisites

Step 1 — Install Claude Code

Step 2 — Choose a project name

Step 3 — Bootstrap a session (auto OR manual)

Path A — auto-bootstrap (recommended)

Path B — pre-create a session manually

Step 4 — Provide credentials to Herald

Step 5 — Verify Dispatch (E18 part A)

Step 6 — Verify session persistence (E18 part B)

Step 7 — (Optional) Test the full vertical slice (E19)

Troubleshooting

Spec + code references

Herald — Claude Code Dispatcher Setup Guide

Field	Value
Status	LIVE (HRD-012 — closed atomically with live E18 evidence captured)
Spec ref	V3 §33 + §33.2
HRD	HRD-012 (Fixed)
Env vars	HERALD_CLAUDE_BIN (default <code>claude</code>), HERALD_CLAUDE_PROJECT_NAME, HERALD_CLAUDE_SESSION_UUID (optional auto-bootstrapped), HERALD_CLAUDE_WORKDIR (optional)
Code	<code>commons_messaging/dispatch/claude_code/</code>

The Claude Code dispatcher is Herald's LLM/agent dispatch surface. It takes an inbound message + context, invokes `claude --resume <session> --print "<envelope>"`, parses the structured `<<<HERALD-REPLY>>>` JSON reply, and returns a `DispatchResponse` for downstream channels.

This guide explains how to set up Claude Code so Herald can dispatch to it.

Table of contents

- [Pre-requisites](#)
- [Step 1 — Install Claude Code](#)
- [Step 2 — Choose a project name](#)
- [Step 3 — Bootstrap a session \(auto OR manual\)](#)
- [Step 4 — Provide credentials to Herald](#)
- [Step 5 — Verify Dispatch \(E18 part A\)](#)
- [Step 6 — Verify session persistence \(E18 part B\)](#)
- [Step 7 — \(Optional\) Test the full vertical slice \(E19\)](#)
- [Troubleshooting](#)
- [Spec + code references](#)

Pre-requisites

- A working Anthropic account with Claude Code access.
- Node.js 18+ available locally (Claude Code is distributed via npm).
- The Herald checkout at `/Users/milosvasic/Projects/Herald`.
- For E18 part B (persistence): container runtime (podman or docker) available.

Step 1 — Install Claude Code

Follow Anthropic's official Claude Code install guide. Quick verification:

```
claude --version
```

Expected: `claude 1.x.y` (or similar — any version that responds with a string is fine).

If `claude` is not on `$PATH` (e.g. you installed to a non-standard location), set `HERALD_CLAUDE_BIN` to the absolute path:

```
export HERALD_CLAUDE_BIN=/Users/you/.local/bin/claude
```

Step 2 — Choose a project name

Herald uses `HERALD_CLAUDE_PROJECT_NAME` to identify the Claude session anchor file. It MUST be a filename-safe identifier — alphanumerics + dashes + underscores. Typically you match it to your repo folder name.

For the Herald repo itself:

```
export HERALD_CLAUDE_PROJECT_NAME=Herald
```

For a consuming project (e.g. ATMOSphere):

```
export HERALD_CLAUDE_PROJECT_NAME=ATMOSphere
```

The session anchor will live at `<workdir>/.herald/claude-code/sessions/<HERALD_CLAUDE_PROJECT_NAME>.session`.

Step 3 — Bootstrap a session (auto OR manual)

Claude Code organizes work into **sessions** identified by UUIDs. Herald's `ResolveSession()` (per spec §33.2) finds or creates the right session automatically — OR you can pre-create one.

Path A — auto-bootstrap (recommended)

Just leave `HERALD_CLAUDE_SESSION_UUID` unset.

When Herald's Dispatch is invoked the first time:

1. `ResolveSession()` looks for the anchor file at `<workdir>/.herald/claude-code/sessions/<project>.session`.
2. If absent (first run), Herald spawns `claude --print "Initializing Herald session for project: <name>"` in the workdir.
3. Claude returns a fresh session UUID in its stdout.
4. Herald captures the UUID and writes it to the anchor file.
5. Subsequent runs read the anchor file and call `claude --resume <uuid>` directly.

This is the §33.2-canonical path. Persistent state lives in the anchor file; the file is per-project and per-workdir.

Path B — pre-create a session manually

For testing, diagnostics, or if you already have a Claude session you want Herald to use:

1. Open Claude Code interactively:

```
claude --print "init"
```

2. Claude returns output that includes a session ID like `3e67dcd3-c66f-4687-b936-562191437310`. Copy it.

3. Export it:

```
export HERALD_CLAUDE_SESSION_UUID=3e67dcd3-c66f-4687-b936-562191437310
```

4. Herald's `ResolveSession()` honors this env var if set — it skips the auto-bootstrap and uses your provided UUID directly.

Step 4 — Provide credentials to Herald

Path A (shell-export, recommended):

```
# Add to ~/.zshrc or ~/.bashrc
export HERALD_CLAUDE_BIN=claude
# leave as 'claude' for PATH lookup
```

```

export HERALD_CLAUDE_PROJECT_NAME=Herald
    # or your project's name
# HERALD_CLAUDE_SESSION_UUID – leave unset for auto-bootstrap
    (Path A in Step 3)
#
    OR set to a specific UUID (Path B)
# HERALD_CLAUDE_WORKDIR=.
    # optional;
    defaults to "."

```

Source the file: `source ~/.zshrc` (or `~/.bashrc`).

Path B (`.env`):

Per `../OPERATOR_CREDENTIALS.md` §“Setting credentials in `.env`” — these env vars do NOT contain secrets (the `claude` CLI handles its own Anthropic auth), so they are safe to put in `.env`. For consistency, follow the same pattern as Telegram credentials.

Step 5 — Verify Dispatch (E18 part A)

The Dispatch integration test exec’s `claude --resume <UUID> --print <envelope>` and parses the structured reply.

Pre-requisite for this test: a Claude session UUID (`HERALD_CLAUDE_SESSION_UUID`). If you took Path A above (auto-bootstrap), run a one-time Path B run first to materialize the session, OR run the test from inside Herald’s repo so the anchor file persists across runs.

```

cd /Users/milosvasic/Projects/Herald
go test ./commons_messaging/dispatch/claude_code/
    -tags=integration -run TestDispatch_LiveClaudeInvocation
    -count=1 -timeout=300s -v

```

Expected (after ~20-30s — Claude’s cold start is non-trivial):

```

--- PASS: TestDispatch_LiveClaudeInvocation (24.24s)
PASS

```

Inside the test output you’ll see Claude’s structured reply parsed: a non-empty Outcome field, a non-empty Summary field. The §107 anti-bluff guard rejects a reply where these fields are empty.

If the test FAILs with no `<<<HERALD-REPLY>>>` marker found in `claude stdout`, the `FormatEnvelope` text isn’t instructing Claude to emit the marker. Inspect `commons_messaging/dispatch/claude_code/claude_code.go` `FormatEnvelope` — it should include the marker template per §33.

Step 6 — Verify session persistence (E18 part B)

The persistence test runs Dispatch AND reads back the `claude_code_sessions` row to verify the session UUID + anchor path + last_response JSONB round-trip correctly through Postgres.

Pre-requisites: container runtime (podman or docker) available — the test boots a Postgres container automatically.

```
go test ./commons_messaging/dispatch/claude_code/
      -tags=integration -run TestDispatch_PersistsSessionState
      -count=1 -timeout=600s -v
```

Expected (~30-40s with container boot):

```
--- PASS: TestDispatch_PersistsSessionState (36.23s)
PASS
```

The §107 evidence chain: the persisted `session_uuid` MUST equal the response's `SessionUUID` exactly; the persisted `anchor_path` MUST equal the response's `AnchorPath`; the persisted `Outcome` MUST equal the response's `Outcome`.

Step 7 — (Optional) Test the full vertical slice (E19)

E19 ties Telegram + Claude Code together: hand-sent Telegram message → Subscribe → Dispatch → reply via Send.

Requires: - All of this guide's env vars set - All of `../messengers/TELEGRAM.md` §3 env vars set - `HERALD_TGRAM_LIVE_INBOUND=1` - Container runtime

Run it per `../messengers/TELEGRAM.md` §6.

Troubleshooting

claude --version reports “Unsupported Node.js version”: Some Claude Code versions have engine-version restrictions. Check the warning message for the supported range and switch Node versions (e.g. via `nvm`) if needed. Currently codegraph 0.8+ on Node 25.x has known incompatibilities; codegraph 0.6.8 works.

Test FAILs with “no conversation found with session ID”: The session UUID you provided doesn't exist in Claude's local session storage. Either: - Bootstrap a fresh session per Step 3 Path B, OR - Remove `HERALD_CLAUDE_SESSION_UUID` from your env and let Herald auto-bootstrap per Step 3 Path A.

Test FAILs with “no <>> marker found”: Claude is not emitting the marker. Either: - The envelope (per `FormatEnvelope` in `claude_code.go`) doesn't instruct Claude to emit it — inspect + fix. - Claude refused the request (e.g. content moderation, rate limit). Inspect the captured stdout — if Claude responded with a refusal, that's a real outcome to handle, not a parse bug.

SQLSTATE 28P01 when running the persistence test: Postgres container's stored password hash doesn't match `HERALD_DB_PASSWORD`. Either reset the volume (`podman-compose down -v` — destroys data) or run `ALTER USER herald PASSWORD '...'` inside the container.

HERALD_CLAUDE_BIN env var has no effect: Confirm with `echo "$HERALD_CLAUDE_BIN"`. If empty, your shell didn't pick up the export. Re-source `~/ .zshrc` or open a new terminal.

Claude session expiry: Claude Code sessions can be invalidated server-side (rare). If `--resume` fails for a session that worked before, re-bootstrap.

Spec + code references

- **Spec:** V3 §33 (LLM/agent dispatch architecture), §33.2 (session-resolution algorithm), §33.3 (response schema)
- **HRD:** HRD-012 (closed atomically per §107 — live PASS evidence captured)
- **Code:**
 - `commons_messaging/dispatch/claude_code/claude_code.go` — Dispatcher struct + New + NewWithStorage + ResolveSession + PersistSession + FormatEnvelope + DispatchResponse + replyJSONSchema + HeraldSystemTenant
 - `commons_messaging/dispatch/claude_code/dispatch.go` — Dispatch (exec.CommandContext `claude -resume`) + parseReply (marker + JSON unmarshal)
 - `commons_messaging/dispatch/claude_code/persist.go` — PersistSessionState (upsert `claude_code_sessions` row)
- **Tests:**
 - `commons_messaging/dispatch/claude_code/dispatch_integration_test.go` — E18 part A (live Dispatch round-trip)
 - `commons_messaging/dispatch/claude_code/persist_integration_test.go` — E18 part B (session persistence with exact-equality assertions)
 - `commons_messaging/vertical_slice_integration_test.go` — E19 full slice
- **Anchor-file format:** a single line containing the UUID. Stored at `<workdir>/.herald/claude-code/sessions/<project>.session`.
- **Related:** [OPERATOR_CREDENTIALS.md](#) §“Claude Code dispatcher”; [../messengers/TELEGRAM.md](#) for the messenger half of the vertical slice